

PRODUCT REQUIREMENTS DOCUMENT

Open-Source Attendance Monitoring & Correction Automation Platform

Local-first • Multi-college • Security-first • GitHub-ready

Version: 1.0

Status: Implementation-ready

Core principle: public source code, private student data, explicit consent, least-privilege integrations, and fail-closed automation.

1. Executive Summary

This project is an open-source, local-first application that helps students monitor attendance on their college portal and, when a student explicitly confirms attendance for the day, identify reliable attendance mismatches and send a correction request to the mapped professor. The code is public on GitHub; credentials, attendance data, browser sessions, and tokens remain local to each student.

The application is reusable across colleges through a portal-adapter architecture. College-specific automation is isolated from the core decision engine so contributors can add adapters without rewriting the product.

V1: local web dashboard, SQLite, Playwright adapters, daily confirmation, subject/professor mapping, correction email, audit logs, secure secret handling, dry-run mode, and macOS scheduling. V1 never modifies attendance records.

2. Goals

- Support many colleges through portal adapters.
- Require explicit daily attendance confirmation before any correction action.
- Read today's attendance after a configured cutoff.
- Act only on reliable explicit ABSENT results.
- Map subjects to professor emails and send a correction request.
- Prevent duplicate notifications.
- Keep credentials and private student data out of GitHub.
- Make security a core architecture requirement.
- Provide dry-run, fake-adapter, and test modes.
- Make new adapter contributions straightforward.

3. Non-goals

- Automatic attendance-record modification.
- Bypassing CAPTCHA, MFA, access controls, or rate limits.
- Central collection of student credentials in V1.
- Cloud SaaS hosting in V1.
- Guaranteeing compatibility with every college.
- Treating UNKNOWN as ABSENT.
- Publishing real credentials, tokens, cookies, databases, or private configuration.

4. Core Workflow

- Install from GitHub.
- Run first-time setup.
- Configure portal adapter and secure authentication.
- Authorize email through OAuth where supported.
- Add subjects and professor mappings.
- Explicitly confirm attendance before the cutoff.
- Scheduler runs after the cutoff.

- Portal adapter checks today's attendance.
- PRESENT → no action.
- ABSENT + confirmation + valid mapping → correction notification.
- UNKNOWN/error/stale/ambiguous → no notification; record diagnostic.
- Deduplicate notifications by date + subject.
- Show audit history in dashboard.

5. Critical Business Rules

- No confirmation → no notification.
- UNKNOWN is never ABSENT.
- Only reliable, date-correct results can trigger notification.
- Valid professor mapping is mandatory.
- Date + subject notification dedupe is mandatory.
- Errors and ambiguous parsing fail closed.
- Never bypass CAPTCHA/MFA/security controls.
- V1 has no portal write operation.
- Dry-run must prevent real email sending.
- Important decisions must be auditable.

6. Architecture

Recommended stack: HTML/CSS/JavaScript local UI; Python backend; SQLite; Playwright; Gmail OAuth/API; macOS launchd. The product is a local web application running on the student's machine.

```
Core ■■■ UI / Setup ■■■ Confirmation & Decision Engine ■■■ SQLite / Audit ■■■ Notifications
■■■ Scheduler ■■■ Portal Adapter Interface ■■■ Generic/Test Adapter ■■■ College Adapter A
■■■ College Adapter B ■■■ Community Adapters
```

Portal adapters implement authentication/session handling, navigation, date selection, attendance extraction, and status normalization. They must return UNKNOWN rather than guess when portal behavior is uncertain.

7. Functional Requirements

- FR-01: first-run setup for portal, email, subjects, mappings, timezone, cutoff, and preferences.
- FR-02: dashboard for confirmation, checks, results, notifications, and errors.
- FR-03: one date-scoped attendance confirmation.
- FR-04: stable portal-adapter contract.
- FR-05: normalize portal output to PRESENT / ABSENT / UNKNOWN.
- FR-06: deterministic decision engine.
- FR-07: email integration with least-privilege authorization.
- FR-08: audit log and run IDs.
- FR-09: safe/idempotent scheduling.
- FR-10: unit, adapter, integration, security, and failure-mode tests.

8. Security & Privacy Flow

Security is a first-class product requirement. The public repository must never become a credential store.

Student ↓ First-run setup ↓ Local secure credential storage / OS Keychain ↓ Runtime-only secret retrieval ↓ Portal / Email API ↓ No secrets in logs, screenshots, Git, or telemetry ↓ Audit only non-sensitive events where possible

Secret rules

- Never hard-code passwords, API keys, OAuth tokens, cookies, or session storage.
- Use .env.example with fake placeholders; real .env is gitignored.
- Prefer macOS Keychain for long-lived secrets.
- Prefer Gmail OAuth instead of storing a Gmail password.
- Never log secrets or authentication headers.
- Never commit Playwright storage state, browser profiles, cookies, or screenshots containing private data.
- Use least-privilege OAuth scopes.
- Keep attendance data local by default.
- Provide revoke/reset actions for integrations.
- Do not transmit attendance data or credentials to a central server in V1.

GitHub safety

- .gitignore must cover .env, DB files, logs, browser state, screenshots, tokens, and local config.
- Add secret scanning/pre-commit checks and CI scanning where practical.
- Test fixtures use fake credentials only.
- Add SECURITY.md and responsible disclosure instructions.
- Tell contributors never to paste secrets into issues or pull requests.
- Review dependencies and pin versions where appropriate.

Threat model

Consider accidental secret commits, malicious dependencies, stolen browser sessions, local malware, portal UI changes, email-token leakage, log leakage, adapter bugs, duplicate notifications, and unintended portal actions. Mitigate through OS credential storage, least privilege, secret scanning, dependency review, fail-closed parsing, adapter isolation, audit logs, dry-run mode, and a read-only V1.

9. Authentication & Email

Portal authentication is adapter-specific. If a portal requires interactive MFA or CAPTCHA, the application must not bypass it; it should support an allowed manual authentication flow or report that manual action is required. Gmail should use OAuth/API authorization with only required scopes. The application must never request a Gmail password.

10. Database

Recommended tables: settings, subjects, professor_mappings, attendance_confirmations, attendance_checks, attendance_results, notification_events, and audit_events. Use schema migrations, transactions, consistent timestamps, and unique constraints. Notification events must be unique for date + subject.

11. UI

- Home: today's confirmation and automation status.
- Attendance: subject status, check time, and reliability.
- Subjects: subject codes/names and professor mapping.
- Email: connected account, authorization, test-email control.
- Portal: adapter, connection test, diagnostics.
- History: confirmations, checks, notifications, failures.
- Security: credential status, revoke/reset controls, privacy information.
- Settings: cutoff, timezone, dry-run, scheduler.

12. Reliability

Decision pipeline: CONFIRMATION → PORTAL HEALTH → DATE VALIDATION → EXTRACTION → NORMALIZATION → RELIABILITY → PROFESSOR MAPPING → DEDUPE → EMAIL → AUDIT. Any failed prerequisite terminates the notification path. Re-running the same date must be safe and idempotent.

13. Testing Strategy

- Unit-test the decision engine independently of Playwright.
- Test PRESENT, ABSENT, UNKNOWN, missing confirmation, missing mapping, duplicate, stale date, timeout, and malformed data.
- Use fake adapters in CI; never use real credentials.
- Test secret redaction in logs.
- Verify dry-run cannot send real email.
- Verify no V1 portal write action exists.
- Run dependency and secret scans.

14. GitHub Structure

```
attendance-automation/ ■■■ app/ ■■■ adapters/ ■ ■■■ generic/ ■ ■■■ / ■■■ tests/ ■■■ docs/
■■■ scripts/ ■■■ .env.example ■■■ .gitignore ■■■ SECURITY.md ■■■ CONTRIBUTING.md ■■■
README.md ■■■ pyproject.toml
```

Each adapter must include documentation, supported portal assumptions, setup requirements, selectors strategy, tests, and a statement that no real credentials/private data are included.

15. Antigravity Development Workflow

- Give Antigravity the PRD and architecture, not secrets in source files.
- Build core + fake adapter first.
- Manually inspect a real portal before implementing selectors.
- Implement the real adapter behind the interface.
- Supply credentials only through local secure runtime configuration.
- Use dry-run during initial real-portal testing.
- Use a controlled recipient before enabling real notifications.
- Review generated code for secrets, network calls, dependencies, and unintended writes.
- Enable scheduling only after controlled testing.

If Antigravity needs real portal access during development, credentials may be supplied through the local runtime mechanism, but they must never be copied into source code, Git history, logs, screenshots, or committed configuration.

16. Example Configuration Contract

```
PORTAL_ADAPTER= PORTAL_URL= PORTAL_USERNAME= PORTAL_AUTH_MODE= EMAIL_PROVIDER=gmail  
DRY_RUN=true CUTOFF_TIME=16:00 TIMEZONE=Asia/Kolkata
```

17. Acceptance Criteria

- Fresh installation works without editing source code.
- Daily confirmation is stored exactly once.
- No confirmation produces no notification.
- PRESENT produces no correction email.
- ABSENT + confirmation + valid mapping produces one eligible correction email.
- UNKNOWN never produces a correction email.
- Repeated runs never duplicate a notification.
- No real secrets appear in repository, tests, logs, or screenshots.
- Dry-run makes decisions without sending real email.
- V1 cannot modify attendance records.
- Security and contribution documentation are present before public release.

18. Release Plan

- Phase 0: security foundation, threat model, secret handling, fake adapter.
- Phase 1: core UI, SQLite, confirmation, decision engine, audit.
- Phase 2: adapter framework and Playwright.
- Phase 3: Gmail OAuth, email template, dedupe, dry-run.
- Phase 4: first real college adapter with controlled testing.
- Phase 5: macOS scheduling and recovery.
- Phase 6: GitHub release, SECURITY.md, CONTRIBUTING.md, CI, scans.
- Phase 7: community portal adapters.

19. Definition of Done

- End-to-end local workflow works.
- Security flow is implemented and documented.
- No real secrets/private student data in repository.
- Portal uncertainty fails closed.
- Email is deduplicated and auditable.
- Fake adapter and dry-run tests pass.
- Controlled real-portal test completed.
- README includes setup, privacy, security, troubleshooting, and adapter contribution guidance.
- V1 is read-only with respect to attendance records.

20. Future Enhancements

- More college adapters.
- Windows/Linux scheduling.
- Desktop packaging.
- Optional encrypted local database.
- Adapter health/change detection.
- Optional local notifications.
- Hosted version only after a separate security/privacy redesign.

21. Core Safety Invariant

CONFIRMED DAY + RELIABLE PORTAL ABSENCE + VALID PROFESSOR MAPPING + NOT ALREADY NOTIFIED = ELIGIBLE CORRECTION NOTIFICATION

Everything else is a fail-closed path.

22. Product Principle

Automate repetitive attendance checking without becoming a credential-collection service or an attendance-modification tool. Public source code, private local data, explicit consent, least privilege, fail-closed decisions, and auditable behavior are the foundations of the project.